Confusion Matrix • From Binary to Multiclass

How do we move from one accuracy number to a detailed picture of model decisions?

PART 1 • BINARY

Start with two classes.

Learn TP, TN, FP and FN and calculate the main classification metrics.

PART 2 • MULTICLASS

Extend the same idea to three or more classes.

Read diagonal and off-diagonal cells.

PART 3 • IRIS

Connect the concept to our Colab workflow with setosa, versicolor and virginica.

Goal: understand not only whether the model is correct, but also how it is wrong.

Why is accuracy not enough?

Accuracy answers one question: “How often was the model correct?” It does not show which kinds of mistakes were made.

MODEL A

86 correct out of 100

Accuracy = 86%

But what were the 14 errors?

MODEL B

86 correct out of 100

Accuracy = 86%

The same accuracy can hide a different error pattern.

A confusion matrix keeps the counts of correct and incorrect decisions separated by actual and predicted class.

Class question: if false positives and false negatives have different consequences, would accuracy alone be sufficient?

Binary classification • The 2 × 2 confusion matrix

	Predicted Negative	**Predicted Positive**
Actual Negative	TN	FP
Actual Positive	FN	TP

ROWS = ACTUAL

What the sample really is.

COLUMNS = PREDICTED

What the model says.

DIAGONAL = CORRECT

TN and TP are correct decisions.

Off-diagonal cells are errors: FP and FN.

TP and TN • Correct decisions

TRUE POSITIVE (TP)

Actual class: Positive
Predicted class: Positive

The model correctly detects a positive case.

Question to ask:
“Was it positive, and did the model say positive?”

TRUE NEGATIVE (TN)

Actual class: Negative
Predicted class: Negative

The model correctly rejects a negative case.

Question to ask:
“Was it negative, and did the model say negative?”

TRUE means the prediction was correct. POSITIVE or NEGATIVE tells us which class the model predicted.

FP and FN • Two different kinds of error

FALSE POSITIVE (FP)

Actual class: Negative
Predicted class: Positive

The model raises a positive prediction incorrectly.

Example: a legitimate email is classified as spam.

FALSE NEGATIVE (FN)

Actual class: Positive
Predicted class: Negative

The model misses a positive case.

Example: a spam email is classified as legitimate.

FALSE means the prediction was wrong. FP and FN are not interchangeable: in many applications their costs are very different.

Worked binary example • 100 test samples

Suppose the positive class is “spam” and the negative class is “legitimate email.”

	Predicted legitimate	**Predicted spam**
Actual legitimate	TN = 54	FP = 6
Actual spam	FN = 8	TP = 32

TOTAL

54 + 6 + 8 + 32 = 100

CORRECT

TN + TP = 54 + 32 = 86

ERRORS

FP + FN = 6 + 8 = 14

Accuracy = 86 / 100 = 0.86 = 86%

Worked example • How do we read each cell?

TN = 54

54 legitimate emails were correctly predicted as legitimate.

FP = 6

6 legitimate emails were incorrectly predicted as spam.

FN = 8

8 spam emails were incorrectly predicted as legitimate.

TP = 32

32 spam emails were correctly predicted as spam.

Always translate the matrix back into the meaning of the classes.

Metrics from the binary confusion matrix

Metric	Formula	Example	Question answered
Accuracy	(TP + TN) / N	86 / 100 = 0.860	How often was the model correct?
Precision	TP / (TP + FP)	32 / 38 = 0.842	When it predicted spam, how often was it right?
Recall	TP / (TP + FN)	32 / 40 = 0.800	Of all spam emails, how many did it find?
Specificity	TN / (TN + FP)	54 / 60 = 0.900	Of all legitimate emails, how many did it reject as non-spam?
F1	2PR / (P + R)	≈ 0.821	How well are precision and recall balanced?

P = precision, R = recall. Which metric matters most depends on the application.

Quick check • Can you identify TP, TN, FP and FN?

Positive class = “fault”. Negative class = “normal”.

	Predicted normal	**Predicted fault**
Actual normal	47	3
Actual fault	5	25

QUESTIONS

1. What is TP?
2. What is TN?
3. What is FP?
4. What is FN?
5. What is the accuracy?
6. Which error means that a real fault was missed?

Suggested answers: TP = 25, TN = 47, FP = 3, FN = 5, accuracy = 72/80 = 90%, missed real fault = FN.

From binary to multiclass

Binary classification has two classes. Our full Iris problem has three.

BINARY

Class 0
Class 1

Confusion matrix size: 2 × 2

TP, TN, FP and FN are directly visible.

MULTICLASS

Setosa
Versicolor
Virginica

Confusion matrix size: 3 × 3

The central idea does not change: rows are actual classes, columns are predicted classes.

What changes is that there are more possible ways for one class to be confused with another.

Multiclass confusion matrix • How do we read it?

Actual \ Predicted	Class A	Class B	Class C
Class A	correct	A → B	A → C
Class B	B → A	correct	B → C
Class C	C → A	C → B	correct

MAIN DIAGONAL

Correct predictions for each class.

OFF-DIAGONAL

Misclassifications between specific pairs of classes.

TOTAL

Every test sample contributes to exactly one cell.

A multiclass confusion matrix shows not only how many errors occurred, but which classes were confused.

Our Colab problem • Three Iris species

CLASS 0 • SETOSA

One of the three target classes.

Target code: 0

CLASS 1 • VERSICOLOR

One of the three target classes.

Target code: 1

CLASS 2 • VIRGINICA

One of the three target classes.

Target code: 2

Inputs X: sepal length, sepal width, petal length, petal width

Target y: Iris species encoded as 0, 1 or 2

This is supervised multiclass classification: the network learns from labelled examples and predicts one of three classes.

Iris example • A 3 × 3 confusion matrix

Illustrative example for reading the matrix. Your exact Colab counts may vary with the trained model and run.

Actual \ Pred.	Setosa	Versicolor	Virginica
Setosa	13	0	0
Versicolor	0	12	1
Virginica	0	1	11

SETOSA

13 correct, no confusion.

VERSICOLOR

12 correct, 1 predicted as virginica.

VIRGINICA

11 correct, 1 predicted as versicolor.

Diagonal total = 13 + 12 + 11 = 36 correct predictions out of 38.

Colab • Generate the multiclass confusion matrix

```
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay

probs = ann.predict(X3_test_s)
y3_pred = probs.argmax(axis=1)

cm3 = confusion_matrix(y3_test, y3_pred)

ConfusionMatrixDisplay(
    confusion_matrix=cm3,
    display_labels=iris.target_names
).plot()
```

1 • PREDICT

The ANN outputs class probabilities.

2 • CHOOSE CLASS

argmax selects the class with the highest probability.

3 • COMPARE

The confusion matrix compares y3_test with y3_pred.

Run this after the 3-class ANN has been trained in the Colab notebook.

Where are TP, TN, FP and FN in a multiclass problem?

For one selected class, convert the multiclass problem into “this class” versus “all other classes.”

EXAMPLE • VERSICOLOR

Positive = versicolor

Negative = setosa + virginica

Now the familiar binary definitions can be used.

TP FOR VERSICOLOR

Actual versicolor predicted as versicolor.

The versicolor diagonal cell.

FN FOR VERSICOLOR

Actual versicolor predicted as setosa or virginica.

Other cells in the versicolor row.

FP = non-versicolor samples predicted as versicolor. TN = non-versicolor samples predicted as another non-versicolor class.

This one-vs-rest view is repeated for each class when class-specific precision, recall and F1 are calculated.

Worked multiclass example • Versicolor as positive

Using the illustrative Iris matrix from the previous slide:

	Predicted non-versicolor	**Predicted versicolor**
Actual non-versicolor	TN = 24	FP = 1
Actual versicolor	FN = 1	TP = 12

PRECISION

Precision = TP / (TP + FP)
12 / (12 + 1) = 0.923

RECALL

Recall = TP / (TP + FN)
12 / (12 + 1) = 0.923

SPECIFICITY

Specificity = TN / (TN + FP)
24 / (24 + 1) = 0.960

The same 3 × 3 matrix can therefore produce class-specific binary metrics for setosa, versicolor and virginica.

Final check • What should we read from a confusion matrix?

1	Where are the correct predictions?	On the main diagonal.
2	What do off-diagonal cells show?	Which classes were confused.
3	What is FP?	A negative sample predicted as positive.
4	What is FN?	A positive sample predicted as negative.
5	How do we extend TP/TN/FP/FN to multiclass?	Choose one class as positive and treat the rest as negative.

Key idea: evaluation is not only “How accurate is the model?” but also “What kinds of decisions and errors does it make?”